



ĐẠI HỌC BÁCH KHOA HÀ NỘI
HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY

KHOA TOÁN – TIN

Lập trình Hướng Đối tượng Object Oriented Programming

Tổng quan





ĐẠI HỌC BÁCH KHOA HÀ NỘI
HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY

KHOA TOÁN – TIN

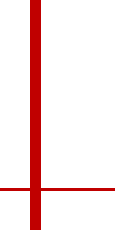
Về học phần



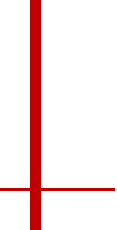
- Tên Lập trình hướng đối tượng
- Mã MI4090
- TC 2-2-0-6
- HP học trước IT1110 - Tin học đại cương
 MI3310 - Kỹ thuật lập trình

- Tên Lập trình hướng đối tượng
- Mã MI4090
- TC 2-2-0-6
- HP học trước IT1110 - Tin học đại cương
 MI3310 - Kỹ thuật lập trình



- 
- Mô tả, nội dung
 - Các tư tưởng lập trình
 - Kiến thức cơ bản về lập trình hướng đối tượng
 - Ngôn ngữ UML
 - Một số mẫu thiết kế
 - Tài liệu





■ Đánh giá

- Quá trình (50%)
 - Các bài tập (theo từng tuần)
 - Chuyên cần, tích cực: ± 2
- Kết thúc HP
 - Báo cáo cuối kỳ

■ Tài liệu

- Lê Đăng Hưng, Tạ Tuấn Anh, Nguyễn Hữu Đức, Nguyễn Thanh Thuỷ (1999). ***Lập trình hướng đối tượng với C++***. NXB Khoa học kỹ thuật
- Mark Allen Weiss (1996). ***Algorithms, Data Structures, and Problem Solving with C++***, Addison-Wesley Publishing Company, Inc.,
- Michael Mcmillan (2007), ***Data structures and algorithms using C#***, Cambridge University Press.
- Dan Clark (2013), ***Beginning C# Object –Oriented Programming***, Apress.
- Erich Gamma, Ralph Johnson, John Vlissides, Richard Helm (1994), ***Design Patterns: Elements of Reusable Object-Oriented Software***, Addison-Wesley Professional.
- Herb Sutter, Andrei Alexandrescu (2004), ***C++ Coding Standards: 101 Rules, Guidelines, and Best Practices***, Addison-Wesley Professional.
- Trịnh Thành Trung (trungtt@soict.hust.edu.vn), Slide LT HDT

- Kết thúc học phần

- Đăng ký nhóm
- Gán chủ đề
- Nộp vào cuối kỳ

- Ngôn ngữ cài đặt: C#, Java, Python

- IDE: VS, VSCode, Eclipse, ...





1. Kỹ thuật lập trình

*Lập trình hướng đối tượng là một kỹ thuật lập trình.
Vậy kỹ thuật lập trình là gì?*

Kỹ thuật lập trình

- Kỹ thuật thực thi một giải pháp phần mềm (cấu trúc dữ liệu + giải thuật) dựa trên nền tảng một phương pháp luận (methodology) và một hoặc nhiều ngôn ngữ lập trình phù hợp với yêu cầu đặc thù của ứng dụng

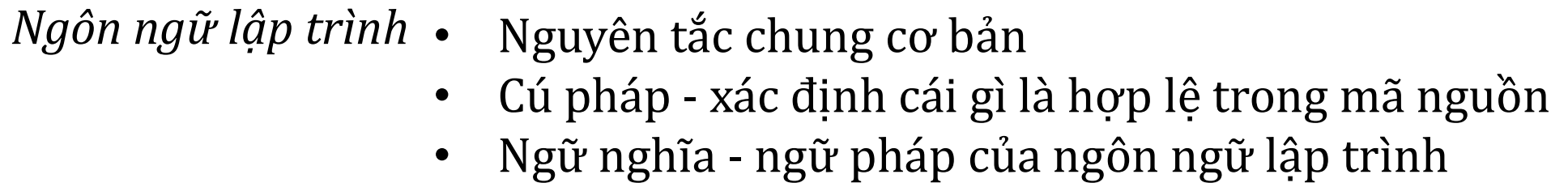


Kỹ thuật lập trình

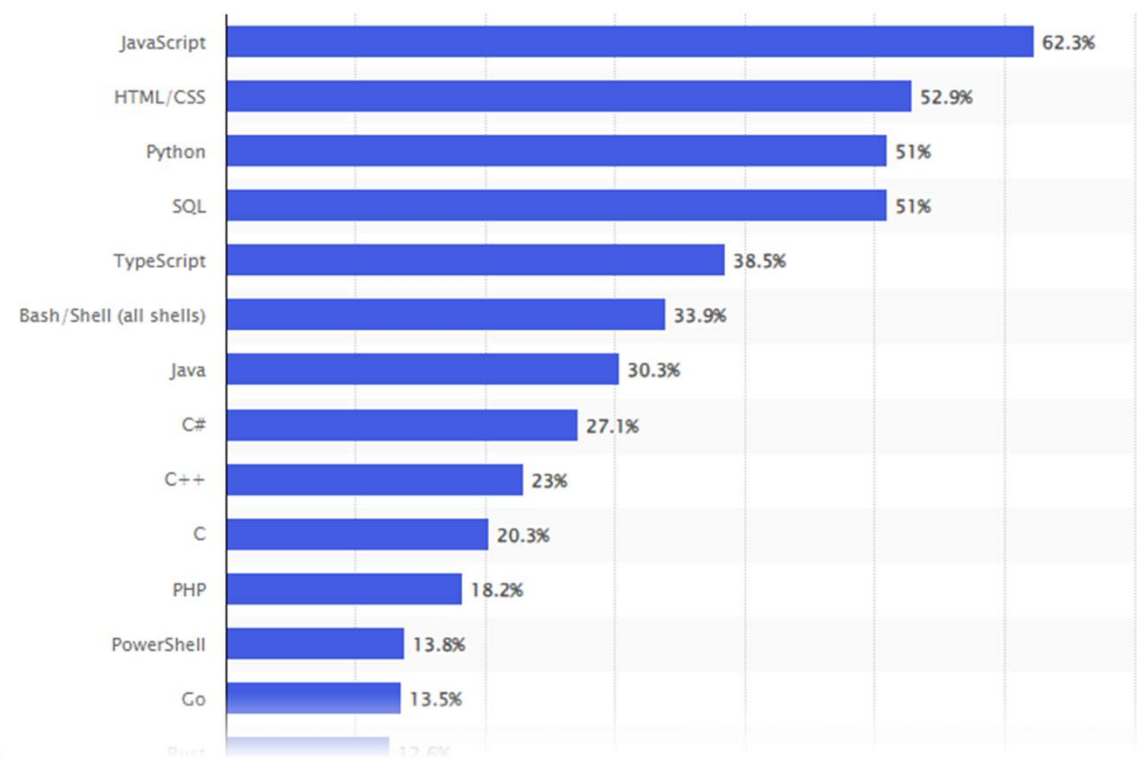
Phương pháp luận

- Các mô thức lập trình (...)
- Các ý tưởng, thuật toán giải quyết vấn đề
- Phong cách trình bày trong lập trình
- Văn hóa lập trình



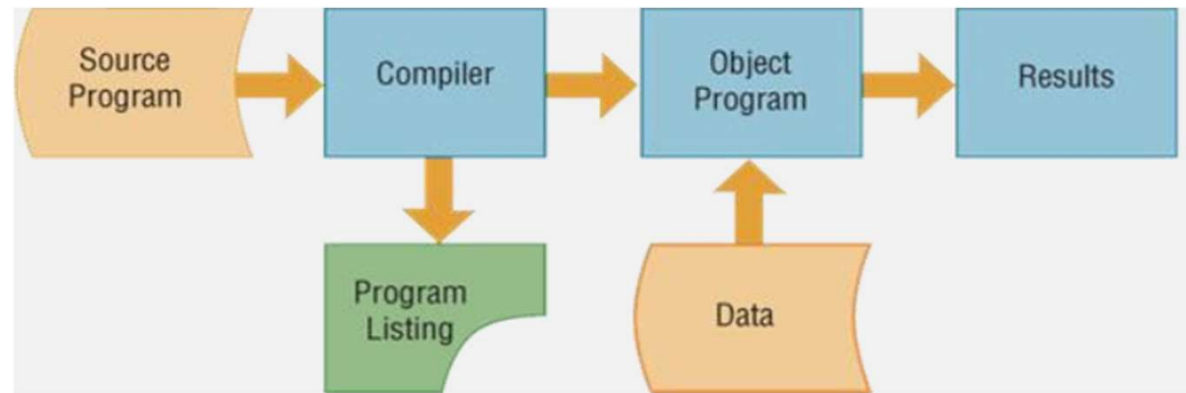


■ Các ngôn ngữ được sử dụng phổ biến



Ngôn ngữ lập trình

- Ngôn ngữ
 - Phương tiện để giao tiếp
 - Hệ thống ký hiệu để diễn đạt
- Từ mã nguồn => chương trình:



Ngôn ngữ lập trình

- Một số thể hệ NNLT
 - Ngôn ngữ máy, Ngôn ngữ gọi nhớ
 - Ngôn ngữ lập trình bậc cao
 - Tuần tự
 - Hướng thủ tục
 - Hướng đối tượng

Lập trình tuần tự

- Mã nguồn được viết dưới dạng "đầu tiên làm thế này, sau đó làm thế kia"
- Khó sửa lỗi, bảo trì.

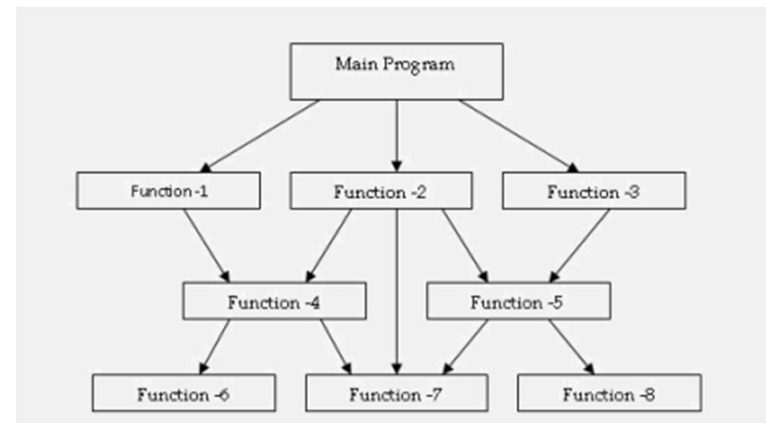
```
05 REM find greatest common divisor (Euclidean
Algorithm)
10 HOME : TEXT
20 LET A = 1071 : LET B = 462
30 IF A < B THEN C = A : A=B : B=C
40 PRINT A,B
50 LET C = A - INT(A/B)*B : REM C = A MOD B (A modulo
B)
60 LET A = B : B = C
70 IF B > 0 GOTO 40
80 PRINT "GCG is ";A
90 END
```

Ngôn ngữ lập trình

- Một số thể hệ NNLT
 - Ngôn ngữ máy, Ngôn ngữ gọi nhớ
 - Ngôn ngữ lập trình bậc cao
 - Tuần tự
 - Hướng thủ tục
 - Hướng đối tượng

Lập trình hướng thủ tục (Lập trình cấu trúc)

- Xây dựng chương trình dựa trên các hàm/thủ tục/chương trình con
- Dữ liệu và xử lý (hàm) tách rời nhau
- Các hàm không bắt buộc phải tuân theo một cách thức chung truy cập vào dữ liệu



Trừu tượng hóa

- Sự phát triển của các ngôn ngữ lập trình chính là sự phát triển của quá trình trừu tượng hóa (abstraction)
 - Ngôn ngữ máy => Assembly
Trừu tượng hóa kiểu dữ liệu/chỉ thị cơ bản
 - Ngôn ngữ lập trình có cấu trúc
Trừu tượng hóa điều khiển (control abstraction), trừu tượng hóa chức năng (functional abstraction)



ĐẠI HỌC BÁCH KHOA HÀ NỘI
HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY

KHOA TOÁN – TIN

2. Kỹ thuật hướng đối tượng



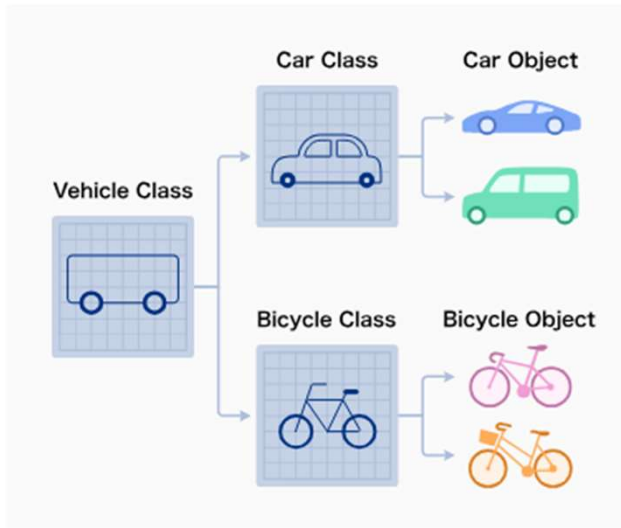
Lược sử

- 1961–1967: Simula - ngôn ngữ HDT đầu tiên
- 1966–1970s: Smalltalk, Alan Kay
- 1980s : xuất hiện nhiều NN HDT (C++, object-C, Lisp, ...)
- 1990s : chiếm ưu thế

Đối tượng

- Khái niệm đối tượng

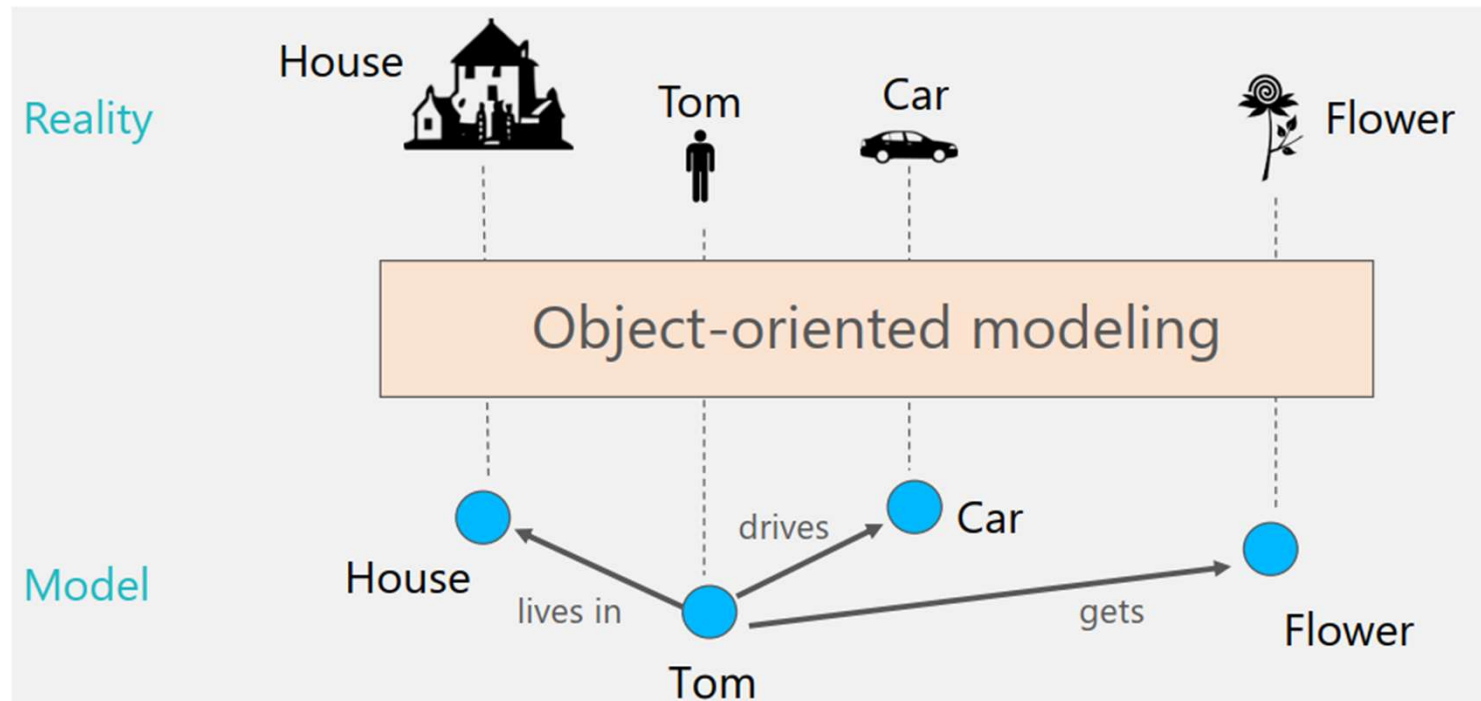
An object is a data structure or abstract data type containing fields and methods



Tất cả mọi thứ đều là đối tượng
- Alan Kay

Lập trình hướng đối tượng

- Trong OOP: Chương trình là một tập hợp các đối tượng tương tác với nhau



Lập trình hướng đối tượng

- Mỗi đối tượng trong chương trình có các dữ liệu độc lập của mình và chiếm bộ nhớ riêng của mình
- Mỗi đối tượng đều có dạng đặc trưng của lớp các đối tượng đó
- Tất cả các đối tượng thuộc về cùng một lớp đều có các hành vi giống nhau

- *Alan Kay*

Lập trình hướng đối tượng

- Sự phát triển của các ngôn ngữ lập trình chính là sự phát triển của quá trình trừu tượng hóa
- Lập trình hướng đối tượng
 - Trừu tượng hóa dữ liệu (data abstraction)
 - Coi các dữ liệu trong chương trình là các đối tượng



ĐẠI HỌC BÁCH KHOA HÀ NỘI
HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY

KHOA TOÁN – TIN

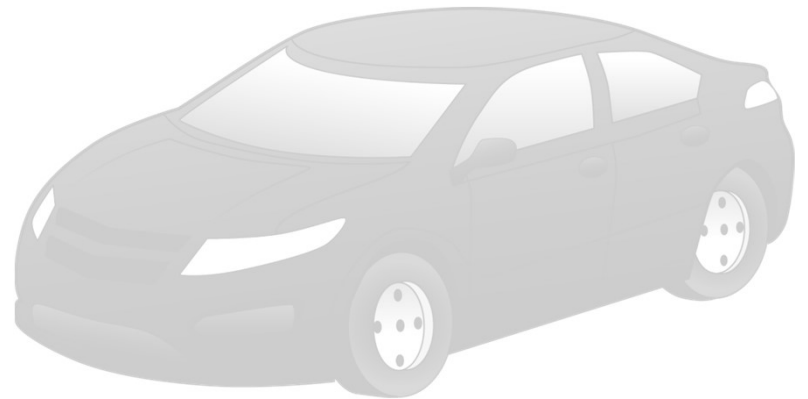
3. Khái niệm cơ bản



Đối tượng

Thế giới thực => mô hình hóa

- Đối tượng trong thế giới thực
 - Sinh viên, giảng viên, ô tô, xe mô tô,...
- Mỗi đối tượng đều có
 - Các thông tin, trạng thái
 - Ví dụ: ô tô có màu sắc, số chỗ, năm sx, ...
sinh viên có ?
 - Các hoạt động
 - Ví dụ: ô tô có thể tăng tốc, phanh, ...
sinh viên có thể ?



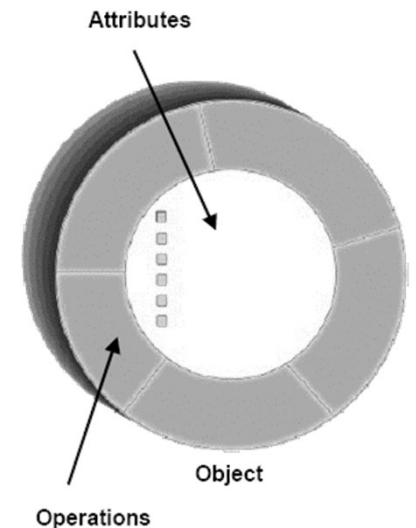
Đối tượng Thể giới thực

	Trạng thái	Hành vi	
Ô tô	Hãng sản xuất Kích thước Màu sắc Số chỗ	Tăng tốc, Giảm tốc	
Tài khoản	Tên tài khoản Số tài khoản Số dư	Rút tiền Gửi tiền Kiểm tra số dư	

Đối tượng

Mô hình hóa thế giới thực

- Mô hình hóa vào trong lập trình: Đóng gói thành trạng thái (state) và hành vi (behavior).
 - Trạng thái được biểu diễn bởi các thuộc tính (attributes) và các mối quan hệ (relationships)
 - Hành vi được biểu diễn bởi các thao tác (operations) hay phương thức (methods)



Trạng thái và hành vi

Trạng thái

- Trạng thái của một đối tượng là một trong các điều kiện tại đó mà đối tượng tồn tại
- Trạng thái của một đối tượng có thể thay đổi theo thời gian



Name: J Clark
Employee ID: 567138
Date Hired: July 25, 1991
Status: Tenured
Discipline: Finance
Maximum Course Load: 3 classes



Professor Clark

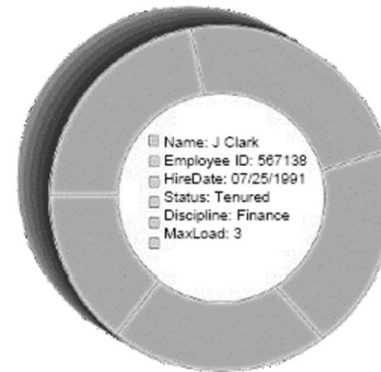
Trạng thái và hành vi

Hành vi

- Hành vi quyết định đối tượng đó hành động và đáp trả như thế nào đối với bên ngoài
- Hành vi nhìn thấy được của một đối tượng được mô hình hóa thành một tập các thông điệp nó có thể đáp trả (các thao tác mà đối tượng đó thực hiện)



Name: J Clark
Employee ID: 567138
Date Hired: July 25, 1991
Status: Tenured
Discipline: Finance
Maximum Course Load: 3 classes



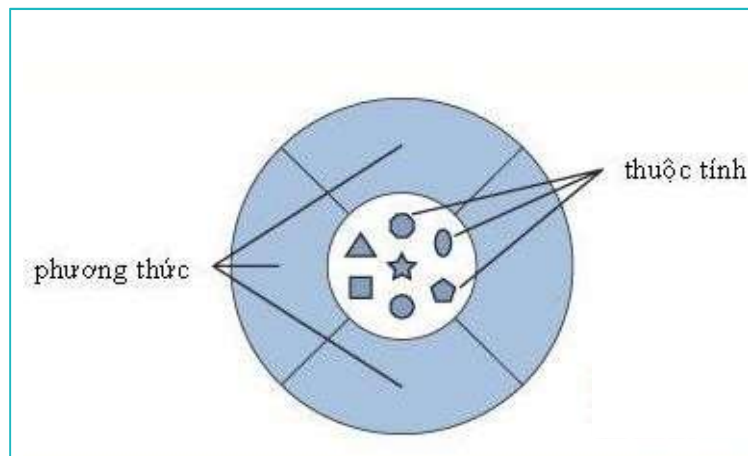
Professor Clark

Đối tượng

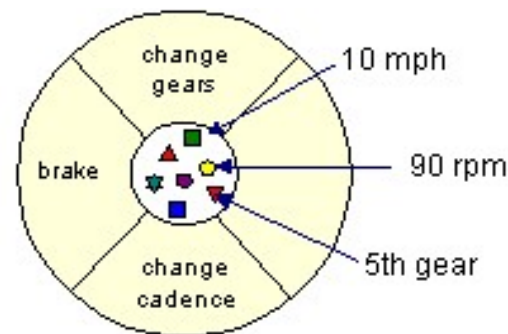
=> Đối tượng trong phần mềm

- Đối tượng (object) là một thực thể phần mềm bao chứa các thuộc tính và các phương thức liên quan
 - Thuộc tính được xác định bởi giá trị cụ thể gọi là **thuộc tính thể hiện**
 - Một đối tượng cụ thể được gọi là một **thể hiện**

Đối tượng phần mềm

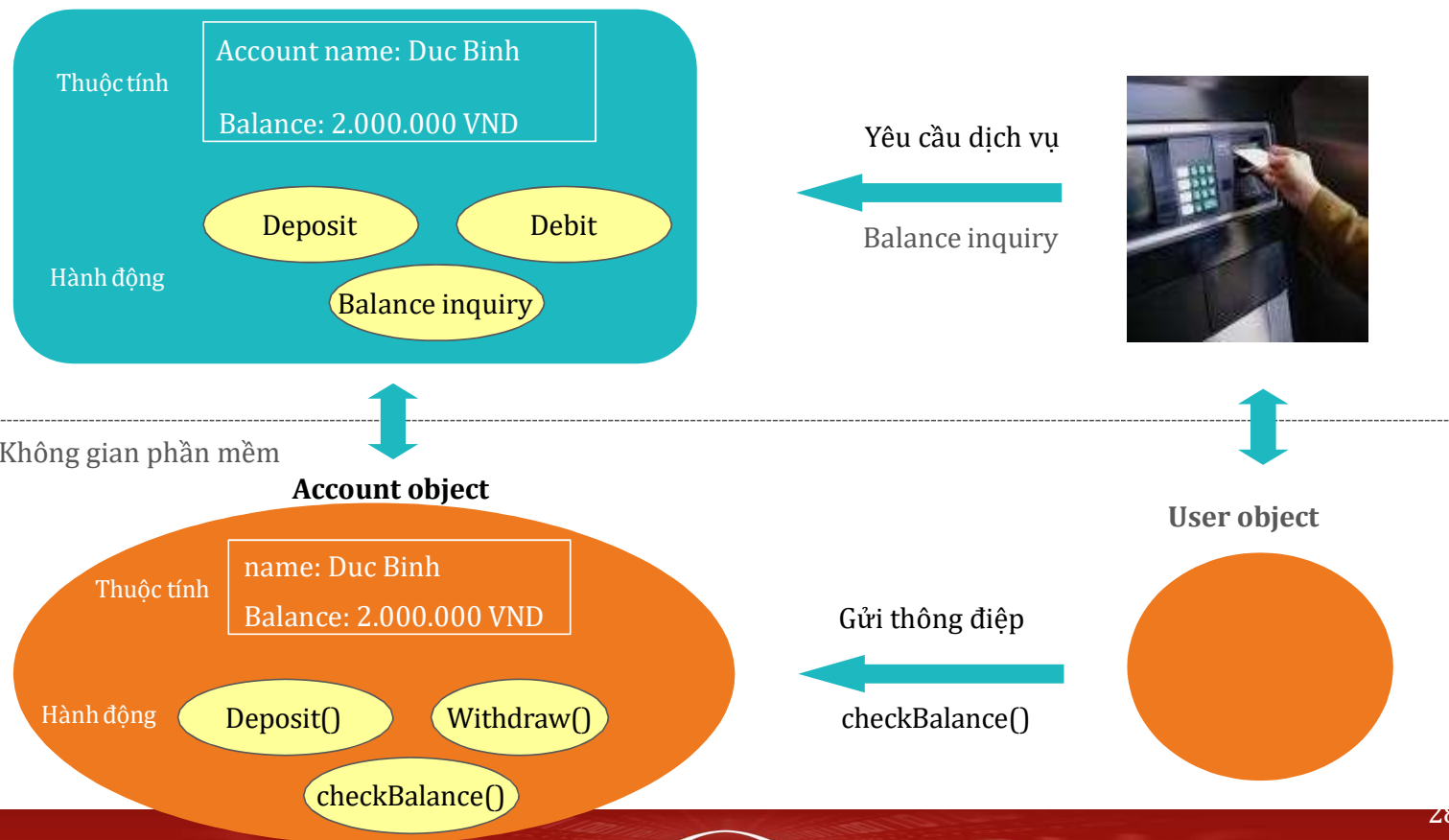


Đối tượng phần mềm **xe đạp**



VD bài toán thực tiễn

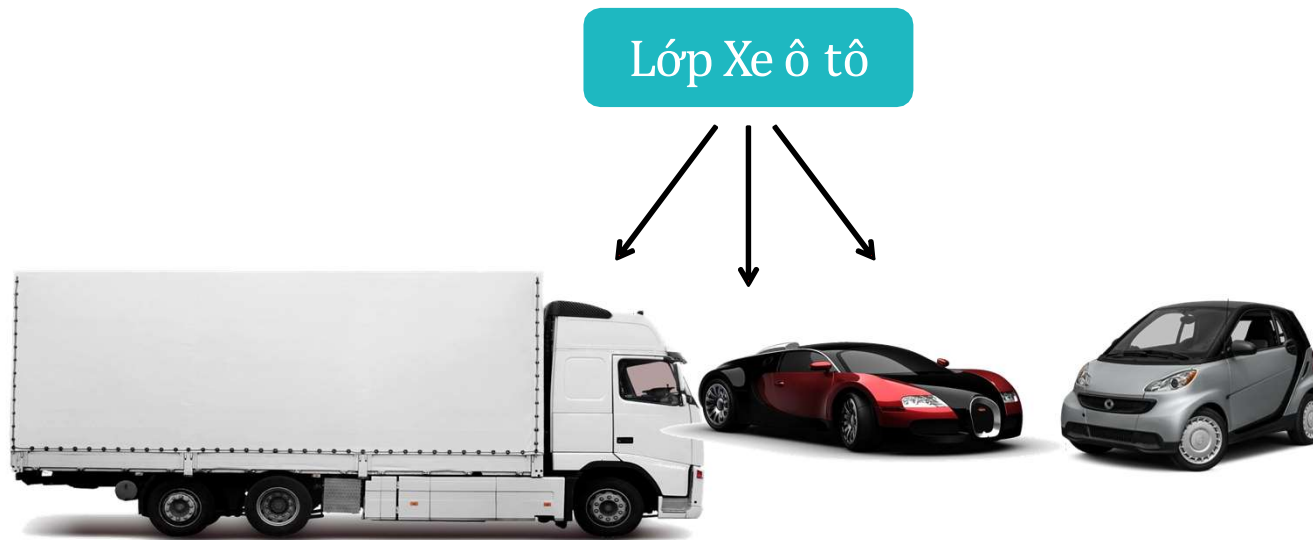
Quản lý tài khoản ngân hàng



Lớp(lớp đối tượng)

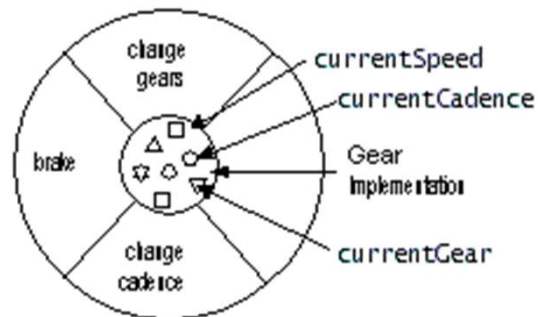
- Một lớp là một thiết kế (blueprint) hay mẫu (prototype) cho các đối tượng cùng kiểu
 - Ví dụ: lớp XeDap là một thiết kế chung cho nhiều đối tượng xe đạp (cụ thể) được tạo ra
- Lớp định nghĩa các thuộc tính và các phương thức chung cho tất cả các đối tượng của cùng một loại nào đó
- Một đối tượng là một thể hiện cụ thể của một lớp.
 - Ví dụ: mỗi đối tượng xe đạp là một thể hiện của lớp XeDap
- Mỗi thể hiện có thể có những thuộc tính thể hiện khác nhau
 - Ví dụ: một xe đạp A có thể có nhiều số trong khi một xe khác có thể có số cố định

Lớp và đối tượng

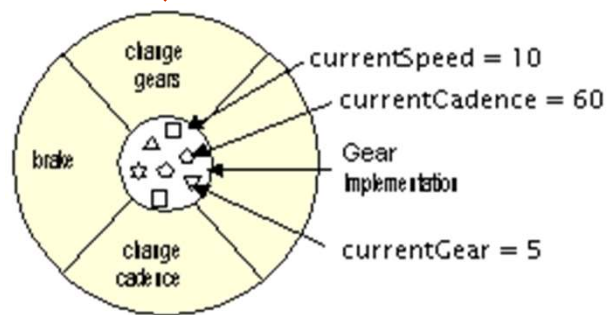


Ví dụ Lớp Xe đạp

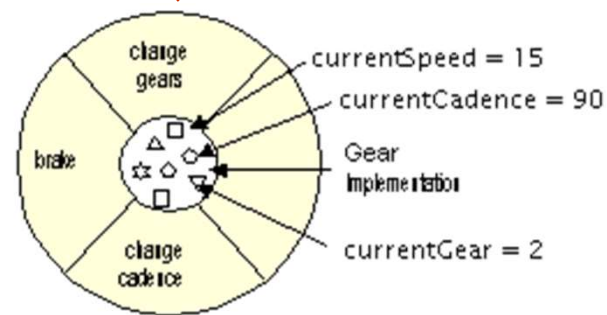
Khai báo cho lớp
XeDap



Đối tượng của lớp
XeDap

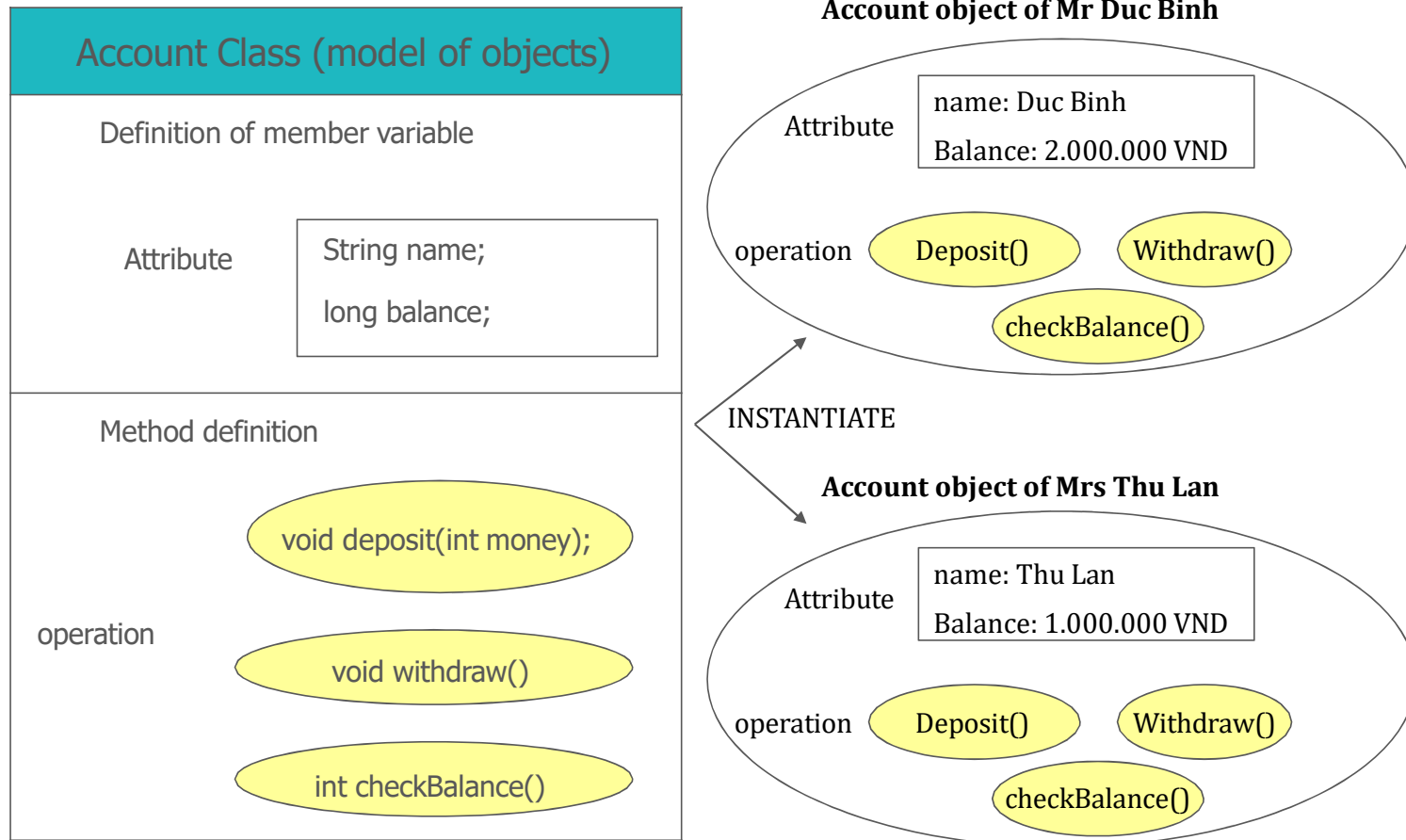


MyBike



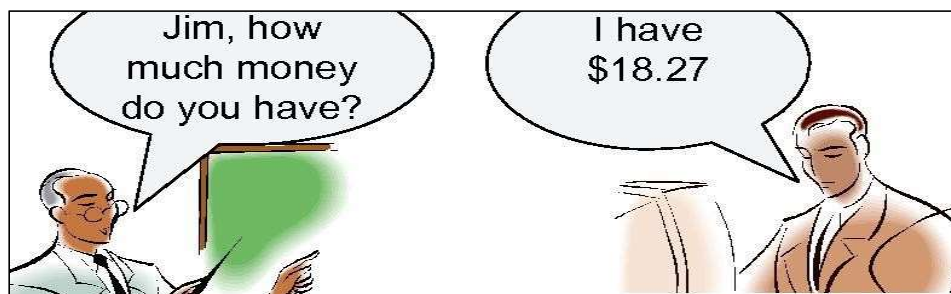
YourBike

Lớp và Đối tượng



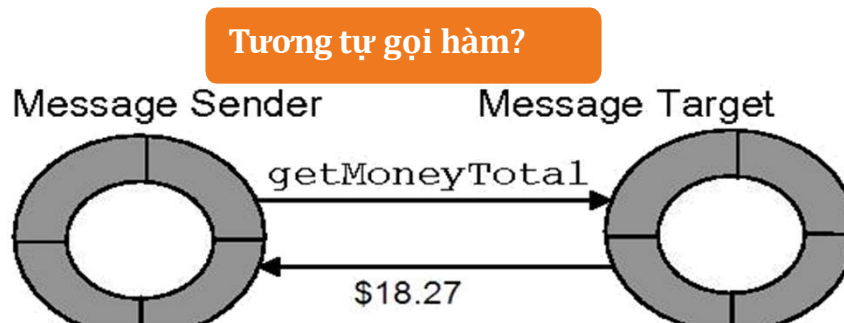
Tương tác giữa các đối tượng

Các đối tượng trong thế giới thực có thể tương tác được với nhau



Trong lập trình:

Các đối tượng giao tiếp với nhau bằng cách gửi thông điệp (message)

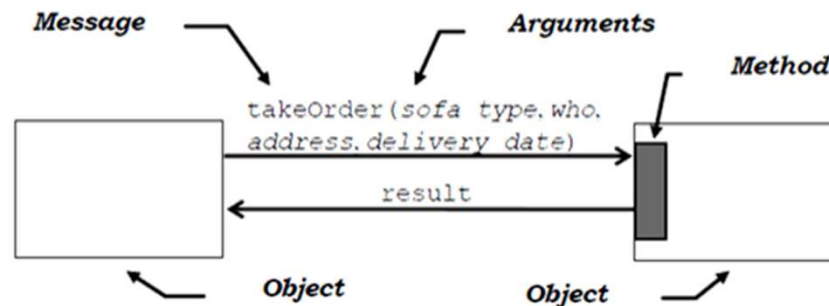


Gọi hàm >< Gửi thông điệp

- Gọi hàm (call function)
 - Chỉ ra chính xác đoạn mã nào sẽ được thực hiện.
 - Chỉ có duy nhất một sự thực thi của một hàm với một tên nào đó.
 - Không có hai hàm trùng tên
- Gửi thông điệp (send message)
 - Yêu cầu một dịch vụ từ một đối tượng và đối tượng sẽ quyết định cần phải làm gì
 - Các đối tượng khác nhau sẽ có các cách thực thi các thông điệp theo cách khác nhau.

Thông điệp vs. Phương thức

- Thông điệp
 - Được gửi từ đối tượng này đến đối tượng kia, không bao gồm đoạn mã thực sự sẽ được thực thi
- Phương thức
 - ~ Thủ tục/hàm trong ngôn ngữ lập trình cấu trúc
 - Là sự thực thi dịch vụ được yêu cầu bởi thông điệp
 - Là đoạn mã sẽ được thực thi để đáp ứng thông điệp được gửi đến cho đối tượng





ĐẠI HỌC BÁCH KHOA HÀ NỘI
HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY

KHOA TOÁN – TIN

4 Các nguyên lý cơ bản

Trừu tượng hóa, đóng gói, module hóa, thừa kế, đa hình

Các nguyên lý cơ bản

LẬP TRÌNH HƯỚNG ĐỐI TƯỢNG

Trừu tượng hóa
(Abstraction)

Đóng gói
(Encapsulation)

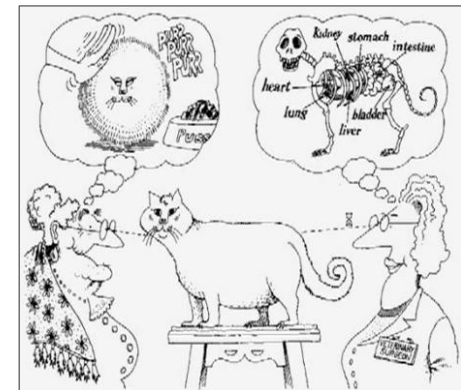
Thừa kế
(Inheritance)

Đa hình
(Polymorphism)

Trừu tượng hóa

■ Trừu tượng hóa

- Là quá trình loại bỏ đi các thông tin cụ thể và giữ lại những thông tin chung
- Tập trung vào các đặc điểm cơ bản của thực thể, các đặc điểm phân biệt nó với các loại thực thể khác
- Phụ thuộc vào góc nhìn
 - Quan trọng trong ngữ cảnh này nhưng lại không có ý nghĩa nhiều trong ngữ cảnh khác



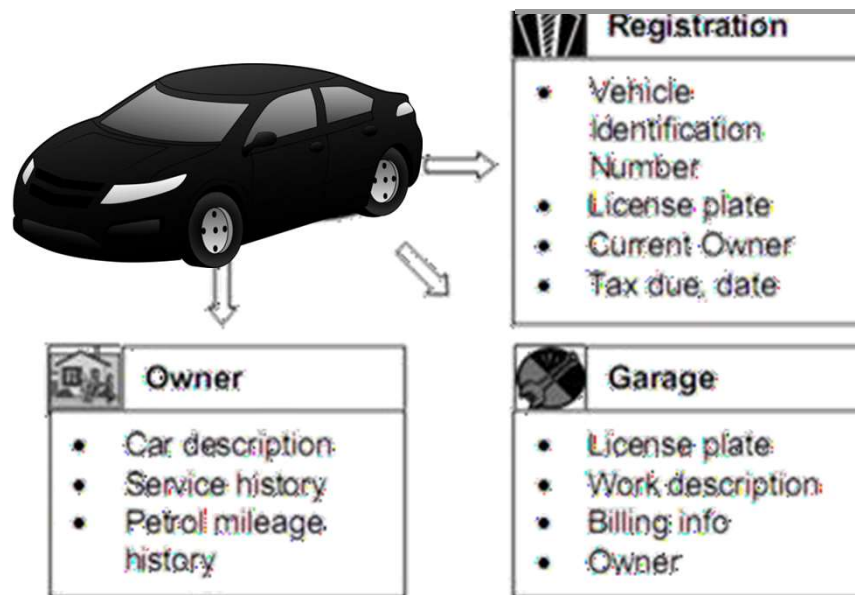


Ví dụ

- Bài toán Quản lý sinh viên

Trừu tượng hóa

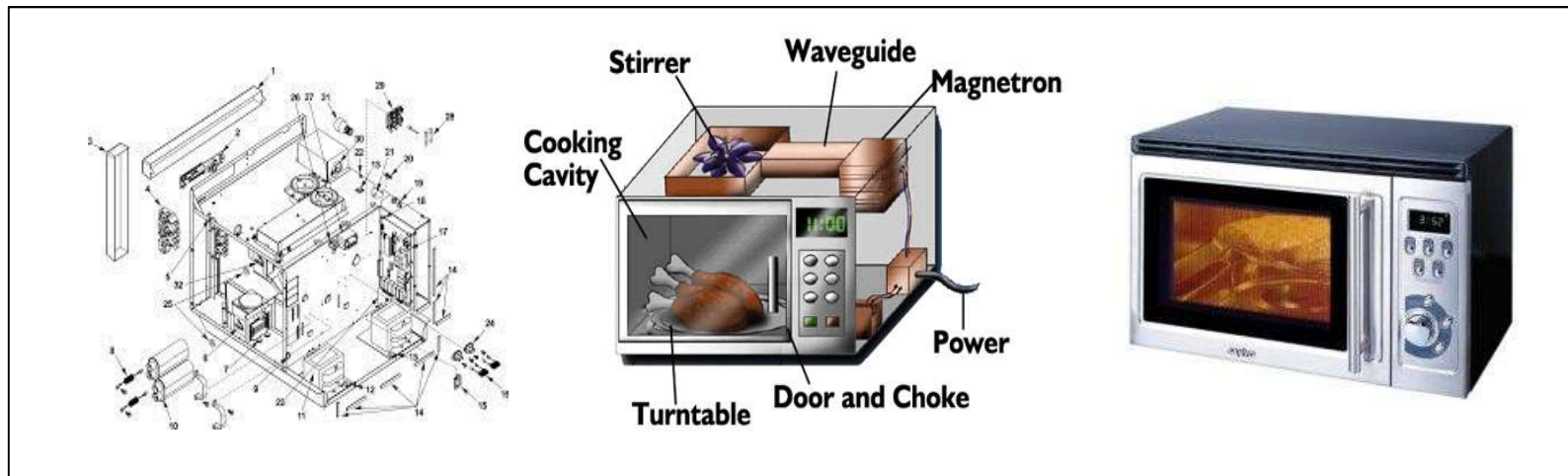
- Phụ thuộc vào góc nhìn:



Đóng gói

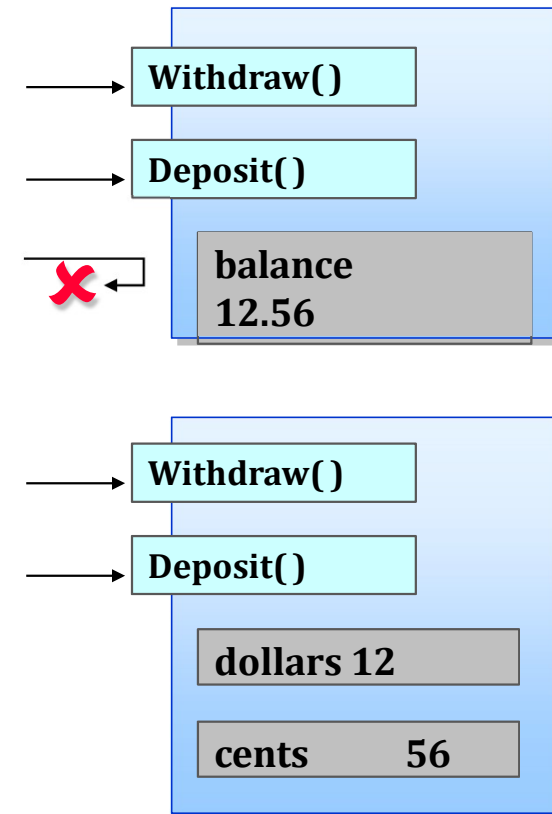
■ Đóng gói

- Che giấu, ẩn đi chi tiết thực hiện bên trong
- Cung cấp cho thế giới bên ngoài một giao diện
- Người dùng không phụ thuộc vào việc sửa đổi sự thực thi bên trong



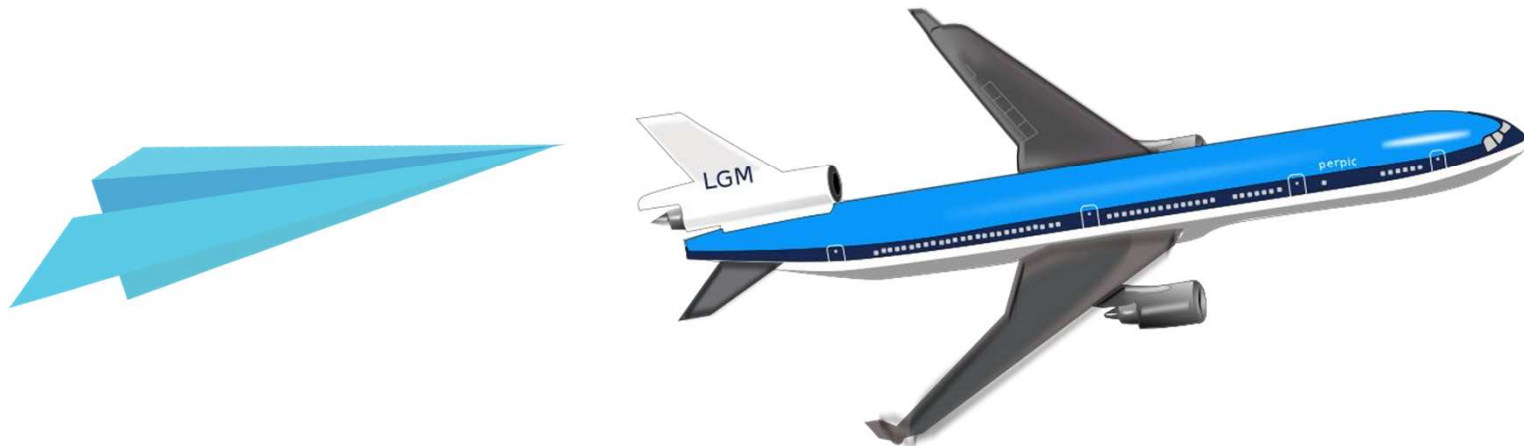
Đóng gói

- Cho phép điều khiển
 - Việc sử dụng đối tượng được kiểm soát thông qua các method public
- Hỗ trợ sự thay đổi
 - Việc sử dụng đối tượng không bị ảnh hưởng nếu dữ liệu nội tại (private) bị thay đổi



Đóng gói

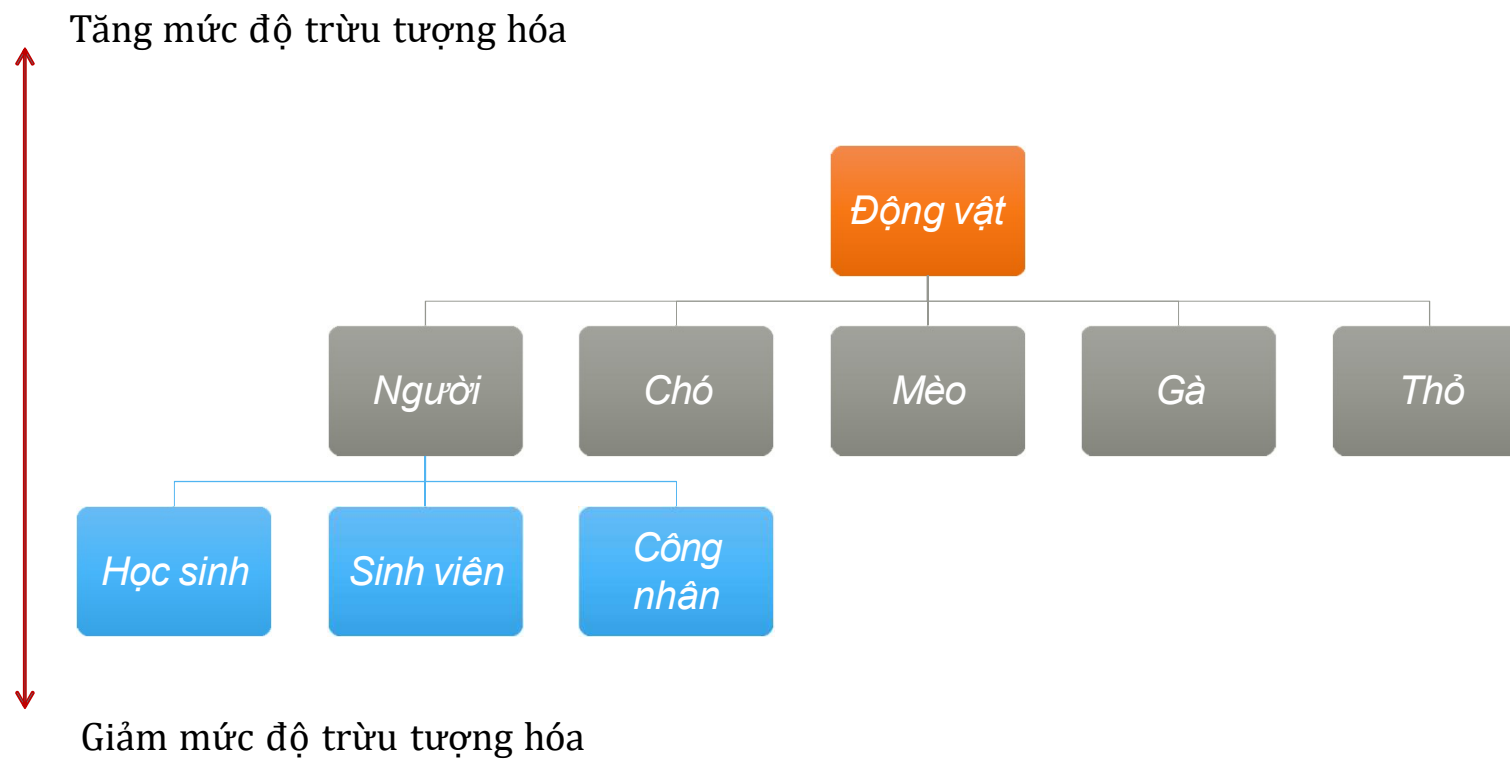
- Cho phép module hóa
 - Chia nhỏ hệ thống phức tạp thành những thành phần nhỏ có thể quản lý được.
 - Cho phép người dùng hiểu được về hệ thống.



Thừa kế

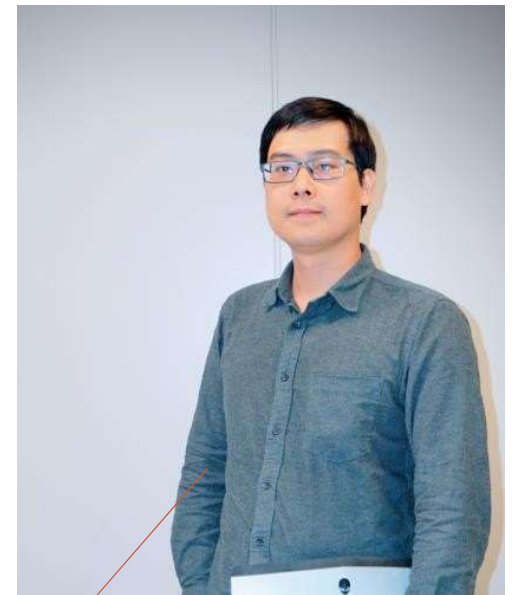
- Phân cấp
 - Xếp hạng hay xếp thứ tự các mức trừu tượng vào một cấu trúc cây
 - Các phần tử ở cùng cấp trong sơ đồ phân cấp thì có cùng mức trừu tượng hóa
 - Tổ chức để phân loại. Sử dụng phân cấp rất dễ dàng nhận ra sự giống và khác nhau giữa các đối tượng

Thừa kế



Đa hình

- Polymorphism: Nhiều hình thức thực hiện, nhiều kiểu tồn tại
- Đa hình trong lập trình
 - Đa hình phương thức:
 - Phương thức trùng tên, phân biệt bởi danh sách tham số.
 - Đa hình đối tượng
 - Nhìn nhận đối tượng theo nhiều kiểu khác nhau
 - Các đối tượng khác nhau cùng đáp ứng chung danh sách các thông điệp
 - có thể giải nghĩa thông điệp theo cách thức khác nhau.



Đối tượng này là giáo viên
hay là lập trình viên?



ĐẠI HỌC BÁCH KHOA HÀ NỘI
HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY

KHOA TOÁN – TIN

5 Phân tích thiết kế hướng đối tượng

Object Oriented Analysis and Design (OOAD)

Phân tích thiết kế hướng đối tượng

- Phương pháp luận (methodology) trong PT&TK phần mềm
- Thông thường được định nghĩa như là một tập các quá trình và thao tác để tìm và khám phá cách có thể giải quyết được bài toán phần mềm.
- Một trong các phương pháp hiệu quả nhất để phát triển phần mềm.

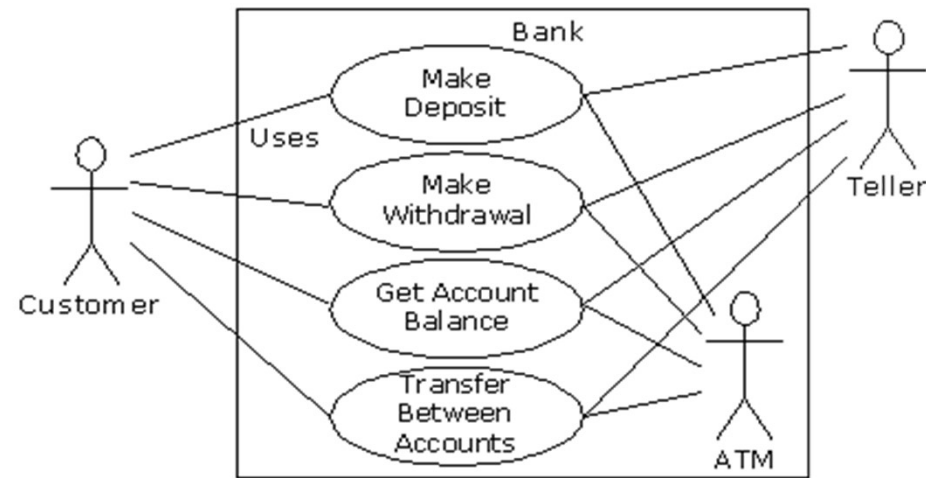
Phát triển phần mềm

- Các giai đoạn cơ bản
 - Giai đoạn 0: Lập kế hoạch (make a plan)
 - Giai đoạn 1: Xác phạm vi, yêu cầu
 - Giai đoạn 2: Phân tích, thiết kế
 - Giai đoạn 3: Cài đặt

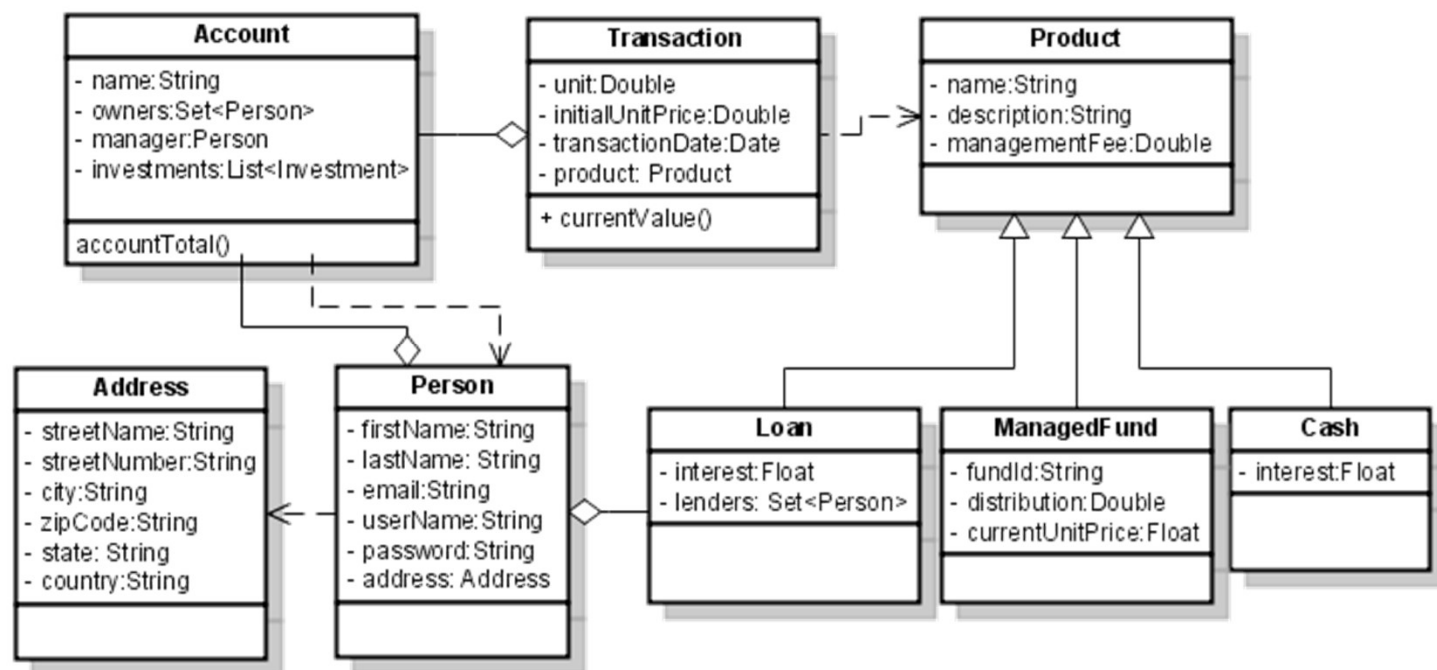
Xác định phạm vi, yêu cầu

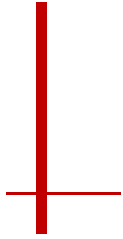
- Xác định mục tiêu - làm gì (what are we making)
- Trong giai đoạn này chúng ta có nhiệm vụ xác định cụ thể các mục tiêu, chức năng và nhiệm vụ mà phần mềm chúng ta cần xây dựng phải đáp ứng
- Trong phương pháp lập trình cổ điển hướng thủ tục người ta gọi giai đoạn này là giai đoạn tạo ra “phân tích yêu cầu và mô tả hệ thống” (requirements analysis and system specification).
- Trong PT&TK hướng đối tượng người ta sử dụng các ký pháp và kỹ thuật Use case để mô tả các công việc này.

Biểu đồ use case



Biểu đồ lớp







ĐẠI HỌC BÁCH KHOA HÀ NỘI
HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY

KHOA TOÁN – TIN

6. Giới thiệu ngôn ngữ C#

Cơ bản về ngôn ngữ

Giới thiệu

- **C# (C-Sharp)** là một **ngôn ngữ lập trình đa mục đích**, được phát triển bởi Microsoft, công bố vào năm 2000
- **.Net** framework
- IDE:
 - VS
 - VS Code



Hello world

- Khai báo Namespace
- Một class
- Phương thức của lớp
- Thuộc tính của lớp
- Một phương thức Main
- Lệnh và biểu thức
- Comment

```
using System;

namespace CS01HelloWorld
{
    class Program
    {
        static void Main(string[] args)
        {
            int input; //câu lệnh
            { //khởi lệnh
                Console.WriteLine("Hello world -xin chào");
            }
        }
    }
}
```

Câu lệnh

- lệnh: <exp>;
- khối lệnh: { }

```
int input; //câu lệnh
{ //khối lệnh
    Console.WriteLine("Hello world -xin chào");
}
```

Biến; hằng số; kiểu dữ liệu và nhập xuất dữ liệu

- Tương tự lập trình thủ tục/phi đối tượng: hỗ trợ biến và hằng số

- Biến

```
// Khai báo biến và gán giá trị
string address = "1 Đại Cồ Việt, Hà Nội, Việt Nam";
// Khai báo biến, sau đó gán giá trị cho biến
int studentAge;
studentAge = 20;
```

- Hằng số

```
const string MON = "THỨ HAI";
```

Biến; hằng số; kiểu dữ liệu và nhập xuất dữ liệu

- C# hỗ trợ sẵn nhiều kiểu dữ liệu, tương đồng các ngôn ngữ thủ tục
 - Số
 - `int` số nguyên có dấu, 32 bit
 - Ký tự, chuỗi
 - `sbyte` số nguyên có dấu, 8 bit; `byte` không dấu, 8 bit
 - `short` số nguyên có dấu, 16 bit; `ushort` không dấu, 16 bit
 - `long` có dấu, 64 bit; `ulong` không dấu, 64 bit
 - Thời gian
 - `float` số dấu chấm động 32 bit; `double` số dấu chấm động 64 bit
 - `decimal` số thập phân, 128 bit
 - Logic ...
 - `char` ký tự 16 bit Unicode;
 - `string` chuỗi ký tự
 - `bool` kiểu logic
 - đối tượng
 - `bool` kiểu logic
 - var
 - `var bien1 = 12;` `var` biến chưa xác định kiểu
 - `object` đối tượng

Biến; hằng số; kiểu dữ liệu và nhập xuất dữ liệu

■ Nhập xuất với Console

- Đối tượng *Console* (System.Console), có phương thức tĩnh để nhập / xuất terminal

- Xuất dữ liệu

Console.Write(value)

Console.WriteLine(value), ...

```
Console.WriteLine("Hello world -xin chào");  
Console.Write("---");  
int input = 10;  
Console.WriteLine("giá trị input={0}",input);
```

- Nhập dữ liệu

v = Console.ReadKey().KeyChar;

v = Console.ReadLine();

```
Console.Write("Nhập họ tên : ");  
string fullname = Console.ReadLine();  
Console.WriteLine($"họ tên: {userLogin}");
```

- Đổi kiểu

Convert.ToInt32(value)

Các phép toán

- Phép toán số học

+ , - , * , / , %

- Gán

= , += , -= , *= , /= , % =

- So sánh, logic

== , >= , <= , != logic: && , || , !

- Điều kiện/rẽ nhánh

if switch rs = expr1 ? expr2 : expr3;

- Vòng lặp

for (; ;) while() do .. while() break continue

foreach (*type* item *in* aCollection)

```
int[] myArray = {1,3,5,19, 10, 20, 40, 40};  
foreach (int element in myArray)  
{  
    Console.WriteLine(element);  
}
```

cấu trúc - struct

■ struct:

- Sử dụng gom các thành phần dữ liệu với xử lý đơn giản
- có thể có các phương thức
- truyền biến: tham trị

• ví dụ đơn giản

```
Product productA;  
productA.name = "Iphone";  
productA.price = 1000;  
Product productB = productA; // gán là sao chép giá trị  
productB.name = "Laptop";  
Console.WriteLine(productA.ToString()); // Iphone : 1000$  
Console.WriteLine(productB.ToString()); // Laptop : 1000$
```

cấu trúc - struct

- struct:

- có thể có các phương thức
- ví dụ

```
public struct Product {  
    public Product(string _name)  
    {  
        name = _name; // khởi tạo thuộc tính Name  
        price = 100;  
    }  
  
    public string name; // trường tên sản phẩm  
    public decimal price; // trường giá sản phẩm  
  
    public override string ToString() => $"{name} : {price}$";  
  
    // Thuộc tính Name  
    public string Name {set => name = value; get => name;}  
    // Thuộc tính Name  
    public string Price {set => price = value; get => price;}  
}  
Product laptop = new Product("dell");
```

liệt kê enum

- khai báo một tập hợp các hằng số có tên, giá trị int mặc định từ 0

```
enum HocLuc {Kem, TrungBinh, Kha, Gioi};  
enum HocLuc {Kem, TrungBinh = 5, Kha, Gioi};
```

- thường dùng với cấu trúc lệnh switch

```
HocLuc hocluc = 0;  
switch (hocLuc) {  
    case HocLuc.TrungBinh:  
        Console.WriteLine ("Học trung bình");  
        break;  
    case HocLuc.Kha:  
        Console.WriteLine ("Học lực Khá");  
        break;  
    case HocLuc.Gioi:  
        Console.WriteLine ("Học lực Giỏi");  
        break;  
}
```

mảng

- Mảng - tập hợp các phần tử (cùng kiểu)

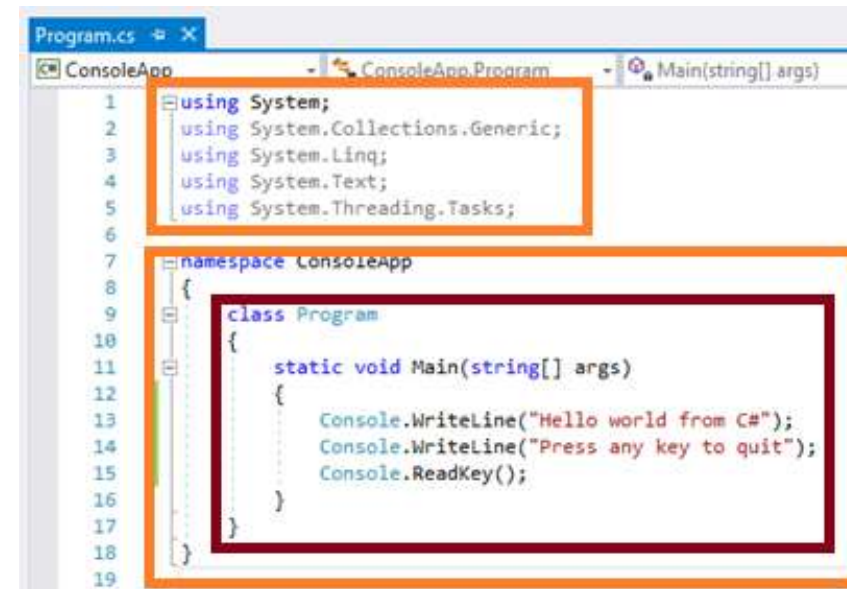
- Thao tác

- khởi tạo
- duyệt
- ...

```
int[] myArray = {1,3,5,19, 10, 20, 40, 40};
string[] studentNames = new string[10];
int maxIndex = myArray.Length - 1;
for (int idx = 0; idx <= maxIndex; idx++) {
    Console.WriteLine(myArray[idx]);
}
foreach (int element in myArray) {
    Console.WriteLine(element);
}
//mảng nhiều chiều: type[, ...,] varname=new type[size1, size2, ..., sizeN];
int[, ] myvar = new int[3,4] {{1,2,3,4}, {0,3,1,3}, {4,2,3,4}};
//mảng trong mảng
int[][] myArray = new int[][] {
    new int[] {1,2},
    new int[] {2,5,6}
};
```

Namespace

- Không gian tên
- Cấu trúc phân cấp các kiểu dữ liệu/đối tượng của C#, .Net => quản lý tốt hơn, tránh xung đột tên
- Đặt tên - định danh (Identifier)
 - không trùng lặp với từ khóa C#
 - bắt đầu bằng một chữ cái, một dấu gạch dưới hoặc sử dụng ký hiệu @



The screenshot shows a C# program in a file named Program.cs. The code is as follows:

```
1 using System;
2 using System.Collections.Generic;
3 using System.Linq;
4 using System.Text;
5 using System.Threading.Tasks;
6
7 namespace ConsoleApp
8 {
9     class Program
10     {
11         static void Main(string[] args)
12         {
13             Console.WriteLine("Hello world from C#");
14             Console.WriteLine("Press any key to quit");
15             Console.ReadKey();
16         }
17     }
18 }
19
```

The code is annotated with boxes: an orange box highlights the 'using' statements (lines 1-5), another orange box highlights the 'namespace ConsoleApp' declaration (line 7), and a red box highlights the 'class Program' and its 'Main' method (lines 9-16).

Chỉ dẫn biên dịch

- Phụ thuộc ngôn ngữ
- Các chỉ thị tiền xử lý (Preprocessor Directive) cung cấp chỉ lệnh tới compiler để tiền xử lý thông tin trước khi sự biên dịch thực sự bắt đầu.
- Tất cả chỉ thị tiền xử lý trong C# bắt đầu với #,

```
#define PI
using System;

static void Main(string[] args)
{
    Console.WriteLine("Chi thi tien xu ly trong C#");
    Console.WriteLine("Vi du chi thi tien xu ly #define");
    #if (PI)
        Console.WriteLine("PI da duoc dinh nghia");
    #else
        Console.WriteLine("PI chua duoc dinh nghia");
    #endif
}
```